

Chiến lược giải các bài toán mở rộng quy mô

Xie Jing

Nguồn gốc: Strategies for solving scaled-up problems

IOI China candidate team papers / 2000 / Xie Jing.pdf

Từ khóa. Mở rộng quy mô; chiến lược; thuật toán.

Tóm tắt

Mở rộng quy mô bài toán là một xu hướng mới trong các kỳ thi tin học gần đây. Nó nhằm tăng độ khó của thiết kế thuật toán và hiện thực chương trình bằng cách tăng lượng dữ liệu, qua đó đặt ra yêu cầu ở tầng cao hơn đối với thí sinh. Bài viết này thử tìm hiểu một số chiến lược có tính phổ quát để giải loại bài toán ấy. Trước hết, bài viết đưa ra định nghĩa của khái niệm “mở rộng quy mô”, rồi dựa vào đó chia thành hai dạng: mở rộng theo chiều ngang và mở rộng theo chiều dọc. Khi bàn về bài toán mở rộng theo chiều ngang, bài viết lấy tư tưởng “hạ số chiều” làm đối tượng chính; còn phần trọng tâm là bài toán mở rộng theo chiều dọc, nơi bài viết nêu hai chiến lược: phương pháp phân rã và phương pháp tinh luyện, sau đó kết hợp với một ví dụ cụ thể để nghiên cứu việc dùng cắt tỉa trong bài toán mở rộng quy mô. Mở rộng quy mô bài toán là một biểu hiện của việc thi tin học tiến gần hơn tới ứng dụng thực tế, nên có ý nghĩa không thể xem nhẹ.

Contents

1	Dẫn nhập	2
1.1	Phân tích bối cảnh	2
1.2	Cách hiểu về “mở rộng quy mô”	2
2	Chiến lược cho bài toán mở rộng theo chiều ngang	3
2.1	Tư tưởng xây dựng chiến lược	3
2.2	Tiểu kết	5
3	Chiến lược cho bài toán mở rộng theo chiều dọc	5
3.1	Phương pháp phân rã	5
3.2	Phương pháp tinh luyện	7
3.3	Dùng khéo cắt tỉa	9
3.4	Tiểu kết	11
4	Kết luận	11

A Phụ lục	11
B Chương trình	12
B.1 Chương trình cho bài toán một	12
B.2 Chương trình cho bài toán hai	13
B.3 Chương trình cho bài toán ba	14
B.4 Chương trình cho bài toán bốn	15
C Tài liệu tham khảo	20

1 Dẫn nhập

1.1 Phân tích bối cảnh

Phân tích đề thi tin học học sinh trung học trong nước và quốc tế những năm gần đây, ta có thể thấy yêu cầu của thi tin học đối với thí sinh không còn chỉ giới hạn ở “thiết kế thuật toán”. Nó đồng thời tăng cường kiểm tra khả năng hiện thực chương trình, chuyển từ chỗ thiên về khảo sát tri thức lý thuyết sang coi trọng cả lý thuyết lẫn thực hành. Sự thay đổi mục tiêu ra đề ấy đã đem lại sức sống mới cho thi tin học và mở thêm một lĩnh vực cho người ra đề. Một biểu hiện của nó là đề thi phát triển từ dạng tĩnh xảo, tức loại mà độ khó chủ yếu nằm ở việc cấu tạo một thuật toán khéo léo, hiểu được mấu chốt là thông suốt, sang dạng quy mô, khiến phần hiện thực bài toán trở nên phức tạp.

1.2 Cách hiểu về “mở rộng quy mô”

Theo từ điển, quy mô là khuôn thức, hình thức hoặc phạm vi mà một sự vật có. Trong thi tin học, quy mô bài toán cụ thể là kích thước lượng dữ liệu cần xử lý, thường có thể biểu diễn bằng một bộ tham số quy mô $(S_1, S_2, \dots, S_k)$. Chẳng hạn, quy mô của bài toán 1 dưới đây là (100), còn quy mô của bài toán 2 là (100, 100).

Bài toán 1. Tính tổng 100 số hạng đầu của một dãy số.

Bài toán 2. Tính tổng mọi phần tử trong một ma trận 100×100 .

Bài toán 3. Tính tổng 1000 số hạng đầu của một dãy số.

“Mở rộng quy mô” nghĩa là mở rộng quy mô của bài toán; cụ thể là tăng số lượng tham số quy mô hoặc mở rộng miền giá trị của các tham số quy mô. Ta biết rằng nếu bỏ qua các yếu tố môi trường như phần cứng và phần mềm máy tính, thì có thể xem khối lượng công việc khi chạy một thuật toán xác định chỉ phụ thuộc vào quy mô bài toán, hay nói cách khác, nó là một hàm của quy mô bài toán. Thời gian chạy và nhu cầu bộ nhớ của chương trình chịu ảnh hưởng trực tiếp từ quy mô ấy. Do nhiều ràng buộc hiện có, khi quy mô mở rộng, tập cách giải thực tế có thể thu hẹp, thậm chí trở thành rỗng; đôi khi điều đó khiến bài toán sau khi mở rộng quy mô không còn giải được bằng mô hình lý tưởng ở quy mô nhỏ ban đầu. Ví dụ bài *Bánh sinh nhật* của NOI 1999 về lý thuyết có thể giải bằng quy hoạch động, nhưng vì chi phí bộ nhớ quá lớn, đa số thí sinh đã hiện thực bằng tìm kiếm.

Từ định nghĩa của “mở rộng quy mô”, không khó thấy nó gồm mở rộng theo chiều ngang và mở rộng theo chiều dọc. Mở rộng theo chiều ngang là tăng số lượng tham số quy mô, chẳng hạn từ bài toán 1 mở rộng thành bài toán 2, tức điều ta thường gọi là đa chiều hóa. Mở rộng theo chiều dọc là mở rộng miền giá trị của tham số quy mô, chẳng hạn từ bài toán 1 mở rộng thành bài toán 3. Các phần sau lần lượt bàn về chiến lược giải nói chung cho hai loại bài toán này.

lại có $s \geq 0$, nên

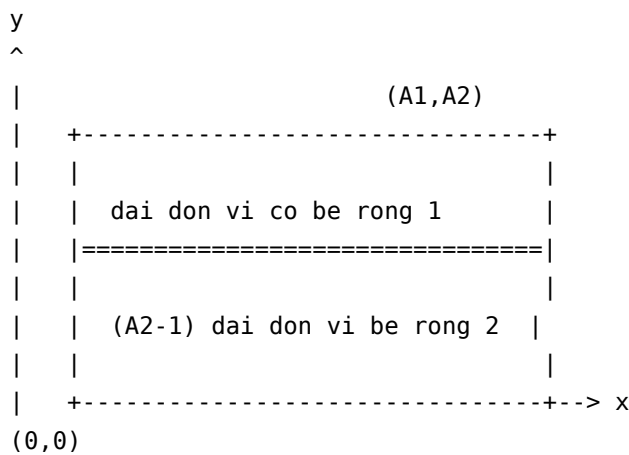
$$0 \leq s \leq A_1 - L.$$

Như vậy có tất cả $A_1 + 1 - L$ đoạn con như thế. Do đó tổng tích bậc của mọi vật thể con một chiều của $((0, A_1))$ là

$$\sum_{i=1}^{A_1} i(A_1 + 1 - i),$$

ký hiệu là $Fg(A_1)$.

Bước 3: mở rộng quy luật sang bài toán nhiều chiều. Ta mở rộng mô hình thêm một chút và xét trường hợp $n = 2$. Khi đó có thể xem vật thể hai chiều là một hình chữ nhật, và tích bậc chính là diện tích hình chữ nhật.



Trong hình trên, ta nhúng hình chữ nhật vào mặt phẳng tọa độ. Ở đây ta thống kê theo chiều dài khác nhau của các hình chữ nhật con, tức khoảng cách trên trục x , và chiều rộng khác nhau, tức khoảng cách trên trục y .

Trước tiên, lấy trong hình chữ nhật một dải hình chữ nhật đơn vị có chiều rộng bằng 1, như phần được đánh dấu trong hình, rồi xét chiều dài của các hình chữ nhật. Theo quy luật khi giải vật thể một chiều, trong dải đơn vị đó có $A_1 + 1 - L$ hình chữ nhật dài L ; vì vậy tổng diện tích mọi hình chữ nhật con trong dải đơn vị là $Fg(A_1)$. Vì trong hình chữ nhật ban đầu có tất cả A_2 dải đơn vị rộng 1, tổng diện tích mọi hình chữ nhật con có chiều rộng 1 là $1 \cdot A_2 \cdot Fg(A_1)$.

Tương tự, tổng diện tích mọi hình chữ nhật con có chiều rộng 2 là $2(A_2 - 1)Fg(A_1)$. Do đó tổng diện tích mọi hình chữ nhật con có chiều rộng W là

$$W(A_2 + 1 - W)Fg(A_1).$$

Suy ra tổng tích bậc của mọi vật thể con hai chiều là

$$Fg(A_2)Fg(A_1).$$

Mở rộng từng bước, ta có tổng tích bậc của mọi vật thể con n chiều của

$$((0, A_1), (0, A_2), \dots, (0, A_n))$$

là

$$Fg(A_1)Fg(A_2) \cdots Fg(A_n),$$

trong đó

$$\begin{aligned}
 Fg(a) &= (1 + 2 + \dots + a) + (1 + 2 + \dots + (a - 1)) + \dots + 1 \\
 &= \frac{1}{2} \{(a + 1)a + a(a - 1) + (a - 1)(a - 2) + \dots + 2\} \\
 &= \frac{1}{2} \{(a + 2)(a + 1)a - (a + 1)a(a - 1) \\
 &\quad + (a + 1)a(a - 1) - a(a - 1)(a - 2) + \dots + 6 - 0\} \\
 &= \frac{1}{6} a(a + 1)(a + 2).
 \end{aligned}$$

Đến đây bài toán được giải trọn vẹn, độ phức tạp thời gian đã giảm xuống $O(n)$, đủ đáp ứng trường hợp số chiều cao.

2.2 Tiểu kết

Dĩ nhiên, đa số bài toán mở rộng theo chiều ngang cuối cùng không thể giải nhẹ nhàng như vậy. Bài toán trong thi thực tế rất phức tạp; ví dụ trên không đụng tới các điểm kiến thức khác là để tập trung minh họa cách vận dụng tư tưởng “hạ số chiều” khi phân tích bài toán. Độ khó của bài toán mở rộng theo chiều ngang chủ yếu nằm ở tư duy, nên ta phải bắt đầu từ những trường hợp đơn giản ở số chiều thấp, tìm quy luật bằng cách khai thác điểm chung giữa bài toán số chiều thấp và số chiều cao. Sau khi tìm được quy luật, không nên mở rộng một cách máy móc sang mô hình nhiều chiều; cần linh hoạt và biết biến đổi để quy luật ấy thật sự phát huy tác dụng.

3 Chiến lược cho bài toán mở rộng theo chiều dọc

3.1 Phương pháp phân rã

Bài toán hai. Tìm số có nhiều ước thực sự nhất trong khoảng từ số nguyên dương N đến M . Trong bài này, ước thực sự được định nghĩa như sau: nếu $R < P$ và R chia hết P , ta gọi R là một ước thực sự của P . Với số nguyên đặc biệt 1, ta xem 1 là ước thực sự của 1.

Phạm vi tham số:

$$1 \leq N < M \leq 999999999, \quad M - N < 999999.$$

Giới hạn thời gian: 10 giây.

Ta rất dễ thu được hai phương pháp sau.

Phương pháp 1: tìm tuần tự. Lần lượt đếm số ước thực sự của từng số nguyên trong phạm vi quy định, rồi ghi nhận nghiệm tốt nhất.

Vì thuật toán phân tích thừa số nguyên tố có độ phức tạp theo bậc căn, độ phức tạp thời gian của thuật toán này là

$$O((m - n)m^{0.5}).$$

Phương pháp 2: đánh dấu. Liệt kê các ước khác nhau và đánh dấu các bội của chúng.

Nếu không phân tích kỹ, ta sẽ nghĩ hai phương pháp có cùng độ phức tạp thời gian. Trên thực tế, độ phức tạp thời gian của phương pháp sau là

$$O\left((m - n) \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{\lfloor m^{0.5} \rfloor}\right)\right),$$

chưa tới $O((m - n) \lceil \log_2 m^{0.5} \rceil)$, trong đó $\lceil x \rceil$ biểu thị số nguyên thuộc khoảng $[x, x + 1)$. Chứng minh như sau.

Trước hết dùng quy nạp toán học để chứng minh

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots + \frac{1}{2^n - 1} \leq n.$$

Khi $n = 1$, vế trái bằng 1, vế phải bằng 1, bất đẳng thức đúng. Giả sử khi $n = k$ mệnh đề đúng, tức là

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots + \frac{1}{2^k - 1} \leq k.$$

Ta chứng minh khi $n = k + 1$, bất đẳng thức vẫn đúng:

$$\begin{aligned} & \frac{1}{2^k} + \frac{1}{2^k + 1} + \frac{1}{2^k + 2} + \cdots + \frac{1}{2^{k+1} - 1} \\ & < \frac{1}{2^k} + \frac{1}{2^k} + \cdots + \frac{1}{2^k} = \frac{2^{k+1} - 1 - 2^k + 1}{2^k} = 1. \end{aligned}$$

Do đó

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{2^k - 1} + \frac{1}{2^k} + \frac{1}{2^k + 1} + \cdots + \frac{1}{2^{k+1} - 1} \leq k + 1.$$

Mệnh đề được chứng minh. Vì vậy

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots + \frac{1}{n} \leq \lceil \log_2 n \rceil.$$

Sở dĩ phương pháp 2 giảm mạnh độ phức tạp thời gian là vì nó dùng mô hình “đổi không gian lấy thời gian”. Trong toàn bộ quá trình đánh dấu, phải lưu số ước thực sự hiện có của mỗi số nguyên, nên độ phức tạp không gian là $O(m - n)$. Từ phạm vi tham số có thể thấy trong thực tế không thể thỏa nhu cầu này. Nó chỉ dừng ở cơ sở lý thuyết, không thể hiện thực thành chương trình. Phương pháp 1 tuy chi phí bộ nhớ nhỏ và có tính khả thi, nhưng chi phí thời gian lại khó đáp ứng yêu cầu. Vì vậy ta có phương pháp sau.

Phương pháp 3: thống kê theo đoạn. Chia khoảng đã cho thành một số khoảng con không trùng lặp và không bỏ sót, rồi thống kê từng khoảng con theo phương pháp 2.

Vì phương pháp 1 mỗi lần xử lý một phần tử đơn lẻ nên tốn thời gian; phương pháp 2 xử lý thống nhất mọi phần tử nên đòi hỏi nhiều bộ nhớ. Phương pháp 3 tổng hợp ưu điểm của hai phương pháp trước: trong khi tận dụng đầy đủ không gian, nó đạt hiệu suất thời gian cao.

Về bản chất, phương pháp 3 chính là một ứng dụng của phương pháp phân rã. Từ đó ta định nghĩa “phương pháp phân rã” như sau:

Lấy một thuật toán làm nguyên mẫu, phân rã bài toán quy mô lớn thành nhiều bài toán con tương đối độc lập, không bỏ sót và cố gắng ít trùng lặp, sao cho hợp của các tập nghiệm của mọi bài toán con chính là tập nghiệm của bài toán ban đầu.

Nguyên lý và phạm vi áp dụng của phương pháp phân rã. Khi giải một số bài toán mở rộng theo chiều dọc, thường xuất hiện “mâu thuẫn” giữa nhu cầu lý thuyết và khả năng chịu đựng thực tế. Mâu thuẫn ấy chủ yếu thể hiện trong quan hệ ràng buộc lẫn nhau giữa nhu cầu thời gian và không gian. Chẳng hạn, quan hệ thời gian–không gian trong bài này có thể biểu diễn bằng một nhánh của hypebol: hai đầu mút của đường cong lần lượt đại diện cho nhu cầu thời gian và không gian của phương pháp 1 và phương pháp 2. Khi ấy, nếu phân rã bài toán thành nhiều bài toán con quy mô nhỏ hơn rồi áp dụng thuật toán cũ để giải, ta có thể trung hòa hiệu quả mâu thuẫn giữa nhu cầu thời gian và không gian. Thông thường,

ta lấy khả năng chịu đựng thực tế về không gian làm tiêu chuẩn chia quy mô bài toán con, để hiệu suất thời gian được nâng cao tối đa. Trong sơ đồ khái niệm dưới đây, đường thẳng đứng biểu thị cận trên của không gian có thể chịu được; giao điểm của nó với đường cong là phương án phân phối nhu cầu thời gian–không gian tốt nhất.

Thời gian

^

```
|  phuong phap 1: chi phi khong gian gan bang 0
| *
| \
| \
| \
| *  phuong phap 2: chi phi thoi gian nho nhat
+-----> Khong gian
|  gioi han khong gian thuc te
```

3.2 Phương pháp tính luyện

Ta định nghĩa “phương pháp tính luyện” như sau:

Bỏ qua các yếu tố bề mặt của bài toán, chỉ rút ra những thông tin đặc biệt có liên hệ thực chất, nhằm tiết kiệm không gian để thích ứng với quy mô của bài toán.

Sau đây dùng một ví dụ cụ thể để minh họa phương pháp giải này.

Bài toán ba: dãy từ dài nhất. Cho một bảng từ tiếng Anh chỉ chứa chữ cái thường. Mỗi từ có ít nhất 1 chữ cái và nhiều nhất 75 chữ cái; bảng được sắp theo thứ tự từ điển tăng dần và không có từ trùng lặp. Tổng số chữ cái của tất cả các từ không vượt quá 2.000.000.

Nếu trong một bảng gồm một hoặc nhiều từ, mỗi từ, trừ từ cuối cùng, đều được từ ngay sau nó chứa, tức là tiền tố của từ sau, thì gọi bảng ấy là một dãy.

Yêu cầu tìm trong bảng từ một dãy từ dài nhất, tức dãy chứa nhiều từ nhất.

Phân tích bài toán. Bản chất của bài toán là: trong một dãy nhất định, tìm dãy con dài nhất sao cho giữa hai phần tử kề nhau, ở đây là các xâu, có một quan hệ đặc thù, tức quan hệ chứa. Khi giải loại bài toán này, ta thường dùng quy hoạch động. Điều đó đã được vận dụng trong bài toán tìm dãy con không giảm dài nhất của một dãy số và bài *Hidden Code* của IOI 1999.

Gọi $num(i)$ là độ dài dãy từ dài nhất có từ thứ i làm đuôi. Khi đó

$$num(i) = \max\{num(j)\} + 1,$$

với $j < i$ và từ j là tiền tố của từ i . Do đó độ phức tạp không gian là $O(n)$, trong đó n là số từ. Lượng dữ liệu của bài này quá lớn, rõ ràng không thể đáp ứng nhu cầu bộ nhớ ấy.

Xét kỹ quan hệ giữa “thứ tự từ điển” và “tiền tố”, ta có thể rút ra kết luận sau: với hai từ w_1, w_2 , nếu w_2 chứa w_1 , và có một từ w_3 thỏa $w_1 < w_3$ và $w_3 < w_2$, thì w_3 nhất định chứa w_1 . Chứng minh ngắn gọn như sau.

Đặt

$$w_2 = L_1 L_2 L_3 \cdots L_{|w_2|}, \quad w_3 = L'_1 L'_2 L'_3 \cdots L'_{|w_3|}.$$

Khi đó $w_1 = L_1 L_2 L_3 \cdots L_{|w_1|}$, trong đó mỗi L_i hoặc L'_i là một chữ cái. Giả sử w_3 không chứa w_1 . Vì $w_1 < w_3$, tất tồn tại một giá trị i , với $i \leq |w_1|$ và $i \leq |w_3|$, sao cho

$$L_j = L'_j \quad (0 < j < i), \quad L_i < L'_i.$$

Khi đó $w_3 > w_2$, mâu thuẫn với $w_3 < w_2$. Vậy giả thiết không đúng.

Vì vậy, mọi tiền tố của một từ nếu tồn tại trong bảng từ thì nhất định được từ đứng ngay trước nó chứa, hoặc chính là từ đứng ngay trước nó. Do đó ta có thể đánh dấu mọi tiền tố của từ, trong đó mỗi từ cũng có thể xem là tiền tố của chính nó, để truyền thông tin cho các từ phía sau. Ví dụ:

i; int; integer; intern; internet

i n t e g e r
i n t e r n
i n t e r n e t

Trước hết, i và int được đánh dấu là tiền tố của từ kế tiếp integer. Khi đọc tới từ intern, ta chỉ cần so sánh integer với intern là thu được dãy tiền tố i và int. Cuối cùng xử lý internet, ta thu được dãy từ dài nhất trong bảng trên là

i; int; intern; internet.

Như vậy, độ phức tạp không gian của bài toán giảm từ $O(n)$ xuống bậc hằng $O(1)$. Ta tiếp tục phân tích độ phức tạp thời gian của hai phương pháp. Nếu lấy phép so sánh giữa hai từ làm thao tác cơ bản, phương pháp quy hoạch động có độ phức tạp thời gian $O(n^2)$, còn phương pháp sau chỉ có độ phức tạp $O(n)$. Ta thấy hai phương pháp không tồn tại quan hệ ràng buộc giữa nhu cầu thời gian và không gian, nên phương pháp phân rã không có chỗ dùng. Mặc dù quy hoạch động đạt hiệu quả tốt trong bài toán tìm dãy con không giảm dài nhất, nó không thích hợp với bài này, chủ yếu vì các lý do sau: lưu trữ xâu phức tạp hơn lưu trữ phần tử của một dãy số, khiến chi phí bộ nhớ lớn; quan hệ chứa giữa các xâu phức tạp hơn quan hệ lớn nhỏ giữa các số, khiến chi phí thời gian lớn; và các phần tử trong bảng từ có tính có thứ tự, cho phép ta dựa vào đặc tính tốt của bài toán để thiết kế thuật toán thích hợp nhất. Phương pháp thứ hai nắm khéo mối liên hệ nội tại giữa từng cặp phần tử, tức các từ, rút ra điểm mấu chốt của bài toán và làm rõ đối tượng xử lý.

Phạm vi áp dụng của phương pháp tinh luyện. Khi đối tượng xử lý chiếm nhiều không gian lưu trữ và thao tác tương đối phức tạp, chẳng hạn thao tác trên xâu, giữa các phần tử như vậy thường tồn tại một số quan hệ đặc biệt. Khi ấy, ta có thể chỉ rút ra mạch chính của bài toán, còn các phần tử thực tế bám trên mạch đó. Nhờ vậy, những phần tử tưởng như rời rạc và lộn xộn được chuyển hóa thành một cấu trúc có quan hệ logic nhất định.

Thật ra, đa số người học không xa lạ với phương pháp tinh luyện. Khi xử lý xâu 01, ta thường chuyển nó thành số thập phân tương ứng, khiến các phần tử rời rạc tạo thành một cấu trúc tuyến tính; đây cũng là một ứng dụng lớn của phương pháp tinh luyện.

Đặc điểm của phương pháp tinh luyện. Phương pháp tinh luyện nằm ở việc nắm bắt bản chất bài toán, nâng quan hệ nội tại giữa các phần tử lên cao hơn, làm mờ đi tính cô lập của từng phần tử, và trừu tượng hóa dữ liệu quy mô lớn thành các quan hệ đơn giản, sáng rõ. Khi đó bài toán dễ mô tả hơn và thao tác thực tế cũng đơn giản hơn. Vì vậy, phương pháp tinh luyện có tính nhắm đích mạnh; khi thiết kế thuật toán, không có khuôn mẫu cố định để áp dụng, mà phải xử lý theo từng bài cụ thể.

3.3 Dừng khéo cắt tỉa

Bài toán thống kê là một thành phần lớn của bài toán mở rộng theo chiều dọc. Vì đa số bài toán loại này có thể giải bằng thuật toán đa thức $O(n^k)$, độ khó không lớn, nên trong thời gian dài chúng khó nổi bật ở các kỳ thi lớn. Gần đây, người ra đề tăng độ khó bằng cách mở rộng quy mô bài toán thống kê, khiến loại bài này bắt đầu xuất hiện sôi nổi trong những kỳ thi tin học quan trọng, chẳng hạn bài *Phủ vệ tỉnh* của NOI 1997 và bài *Chu vi hình* của IOI 1998.

Cắt tỉa là dùng một phép phán đoán nào đó để loại bớt một số quá trình tính toán không cần thiết. Nó bắt nguồn từ thuật toán tìm kiếm; một điều kiện cắt tỉa tốt thường có thể nâng cao rất mạnh hiệu suất thời gian của tìm kiếm. Tuy nhiên, vì thuật toán đa thức thường được xem là thuật toán hiệu quả, rất ít người quan tâm tới điều kiện cắt tỉa, thậm chí cho rằng nó không có tác dụng lớn. Vậy ứng dụng cắt tỉa trong bài toán thống kê rốt cuộc có tác dụng đến mức nào? Ta vẫn nên bắt đầu bằng một ví dụ cụ thể.

Bài toán bốn: Airport của IOI 1999. Tóm lược đề: trong một ma trận số, tìm hình chữ nhật con có diện tích lớn nhất sao cho hiệu giữa số lớn nhất và số nhỏ nhất trong đó không vượt quá ngưỡng C .

Phương pháp 1. Ghi lại với từng hàng độ dài lớn nhất của vùng tuyến tính có cận dưới cố định và chứa phần tử đầu, rồi tìm vùng có diện tích lớn nhất với cận dưới cố định.

Để tiện bàn luận, trong các sơ đồ, hàng và cột được xếp theo thứ tự từ trên xuống dưới, từ trái sang phải. Xét hình chữ nhật 4×4 dưới đây với $C = 4$:

38	38	33	39
39	40	39	39
39	40	41	38
36	39	39	39

Bắt đầu từ ô $(1, 1)$, vùng tuyến tính có cận dưới 36, tức cận trên là $36 + C$, có thể kéo dài đến ô $(1, 2)$. Tương tự, bắt đầu từ các ô $(2, 1)$, $(3, 1)$, $(4, 1)$, vùng tuyến tính có cận dưới 36 lần lượt có thể kéo dài đến $(2, 4)$, $(3, 2)$, $(4, 4)$. Vì vậy, vùng diện tích lớn nhất có cận dưới 36 là $(1, 1, 4, 2)$.

Ta chú ý rằng khi thống kê vùng tuyến tính lớn nhất của từng hàng, vì vùng phải chứa phần tử đầu nên giá trị cận dưới và phần tử đầu không chênh quá C . Do miền giá trị của C là $[0, 10]$, nhu cầu bộ nhớ có thể đáp ứng, nên thuật toán này có tính khả thi. Độ phức tạp thuật toán là $O(uv^2C)$. Vì cận trên của u, v là 700, dù đề cho giới hạn thời gian tối đa tới một phút, chi phí thời gian này vẫn không đáp ứng được.

Phương pháp 2. Nén ngang ma trận để thu được một loạt vùng đơn vị, rồi tìm vùng đơn vị liên tiếp dài nhất.

Ta có thuật toán sau:

1. Xác định biên tây.
2. Xác định biên đông, nhỏ hơn biên tây cộng 100.
3. Thống kê từng vùng đơn vị từ biên tây đến biên đông.
4. Xác định biên bắc.
5. Tìm biên nam nhỏ nhất.

Có thể thấy độ phức tạp thời gian của thuật toán trên là $O(100uv^2)$.

Nếu chỉ xét độ phức tạp thời gian, phương pháp trước chắc chắn tốt hơn phương pháp sau. Nhưng nếu thêm điều kiện cắt tỉa tốt vào phương pháp sau thì kết quả sẽ khác.

Vì ta tìm vùng hình chữ nhật có diện tích lớn nhất và liên tục cập nhật giá trị tốt nhất hiện thời, nên nếu diện tích lớn nhất mà một lần tính toán nào đó có thể đạt được không vượt quá giá trị tốt nhất hiện thời, phép tính ấy không có ý nghĩa và có thể bỏ qua.

Vẫn xét hình chữ nhật 4×4 ban đầu. Khi biên tây là 1 và biên đông lần lượt là 2, 3, 4, ta thu được các vùng diện tích lớn nhất tương ứng. Có thể thấy hiệu giữa biên nam và biên bắc có xu hướng giảm dần. Thực ra điều này không phải ngẫu nhiên: có thể chứng minh bằng phản chứng rằng khi biên tây cố định và biên đông mở rộng dần, tức chiều rộng hình chữ nhật tăng, thì độ dài của vùng hình chữ nhật diện tích lớn nhất là không tăng.

Gọi $Max_wide[wno, eno]$ là độ dài vùng lớn nhất khi biên tây và biên đông lần lượt là wno và eno . Khi đó

$$Max_wide[wno, eno] \leq Max_wide[wno, eno - 1].$$

Vì vậy ta thu được hai điều kiện cắt tỉa sau:

```
If max_width (Min{100,u}) * Max_width[wno,eno-1] < current_best_area
Then expand the west boundary;
```

```
If (eno-wno+1) * Max_width[wno,eno-1] < current_best_area
Then expand the east boundary;
```

Sau đây là đối chiếu thời gian kiểm thử của ba chương trình: `land_1.pas` cho phương pháp 1, `land_2.pas` cho phương pháp 2, và `land_3.pas` cho phương pháp 2 sau khi kết hợp điều kiện cắt tỉa:

Dữ liệu	Quy mô (U, V, C)	Land_1.pas	Land_2.pas	Land_3.pas
1	20, 20, 0	0.05s	0.05s	< 0.05s
2	30, 41, 5	0.05s	0.49s	< 0.05s
3	100, 100, 2	0.22s	0.94s	0.11s
4	100, 500, 7	1.26s	2.20s	0.90s
5	300, 300, 10	9.23s	10.33s	2.47s
6	500, 400, 10	20.99s	98.52s	7.25s
7	600, 500, 5	1.54s	25.73s	4.89s
8	700, 700, 9	99.89s	56.87s	9.78s
9	700, 700, 10	168.85s	1695.93s	34.12s

Môi trường chạy: Pentium 166 MHz, 16 MB. Ghi chú: dữ liệu được sắp theo quy mô từ lớn đến nhỏ.

Phân tích kết quả kiểm thử.

1. Ngoại trừ thời gian chạy của dữ liệu 7 không tương xứng với quy mô dữ liệu, thời gian chạy của chương trình 1 tương đối ổn định theo quy mô. Nguyên nhân chính là trong dữ liệu 7, chênh lệch độ cao giữa các ô vuông kề nhau đa phần lớn hơn ngưỡng C . Vì vậy $O(uv^2C)$ là độ phức tạp thời gian trung bình của thuật toán này và phản ánh khá chính xác chi phí thời gian của nó.
2. Với các bộ dữ liệu có quy mô tăng dần, thời gian chạy của chương trình 2 dao động rất lớn, đặc biệt là dữ liệu 8 và dữ liệu 9: hai bộ dữ liệu có quy mô gần như nhau nhưng kết quả chạy lại khác nhau rất xa. Điều này cho thấy $O(100uv^2)$ chỉ là độ phức tạp thời gian trong trường hợp xấu nhất của thuật toán. Vì chi phí thời gian thực tế của chương trình 2 phụ thuộc mạnh vào dữ liệu, khó dùng độ phức tạp thời gian để phản ánh chính xác.

Phân tích hiệu quả cắt tỉa. Một khi xác định điều kiện cắt tỉa, mỗi lần thống kê đều phải thực hiện một phép phán đoán; vì vậy cắt tỉa không chuẩn xác sẽ đem lại tác động tiêu cực lớn. Hơn nữa, hiệu quả của điều kiện cắt tỉa thường bị cho là có nhiều yếu tố tình cờ, nên nó thường bị lạnh nhạt.

Trong bảng trên, so với chương trình 2, tốc độ chạy của chương trình 3 tăng rất mạnh và nhìn chung tốt hơn chương trình 1. Đối chiếu này cho thấy điều kiện cắt tỉa vẫn có tác dụng lọc bỏ rất tốt. Từ thời gian chạy của nó, ta còn có thể đại khái nhìn ra quy mô dữ liệu. Dù không thể ước lượng chính xác hiệu quả của điều kiện cắt tỉa là bao nhiêu, tính ổn định tương đối ấy đủ cho thấy nếu dùng điều kiện cắt tỉa tốt, hoặc kết hợp nhiều mặt của cắt tỉa, thì lợi ích tốt gần như là tất yếu trong cái tưởng như tình cờ. Cắt tỉa tuy không làm giảm độ phức tạp thời gian của thuật toán, nhưng có tác dụng không nhỏ đối với việc giảm chi phí thời gian thực tế. Đặc biệt với bài toán mở rộng quy mô, chi phí thời gian lớn; nếu chú ý phân tích bài toán và tìm được thông tin ràng buộc, chắc chắn sẽ đạt hiệu quả rất cao.

3.4 Tiểu kết

Vì bài toán mở rộng theo chiều dọc có tính linh hoạt rất lớn, khó tổng kết một tư tưởng hoạch định chiến lược thống nhất như với bài toán đa chiều hóa, và cũng khó quy nạp thành một tập chiến lược tương đối hoàn chỉnh. Những điều trên chỉ là một số chiến lược mà tôi tự tổng kết từ kinh nghiệm luyện tập thường ngày và cho rằng có ý nghĩa mở rộng nhất định. Trong đó cũng nhắc tới ứng dụng của cắt tỉa trong bài toán mở rộng theo chiều dọc; tuy nó không phải một chiến lược cụ thể, nó vẫn có ý nghĩa phổ quát đối với bài toán mở rộng quy mô, như đã đề cập ở trên. Tóm lại, mục đích nghiên cứu các chiến lược là nhất quán: giải bài toán một cách hiệu quả. Tin rằng thông qua học tập liên tục, đồng thời trao đổi và thảo luận với các bạn học cũng như huấn luyện viên, ta nhất định sẽ khám phá được nhiều chiến lược hơn và có giá trị hơn.

4 Kết luận

Ta biết rằng dù thuật toán được tối ưu hóa thế nào, quy mô bài toán có thể giải cuối cùng vẫn hữu hạn. Vì vậy, khi giải bài toán mở rộng quy mô, trước hết phải làm rõ quy mô thực tế của bài toán, rồi “đo ni đóng giày” để thiết kế thuật toán phù hợp. Đôi khi đề bài còn thêm một số điều kiện hạn chế đối với quy mô bài toán. Chẳng hạn trong bài *Hidden Code* của IOI 1999, độ dài tối đa của văn bản là 1.000.000, nhưng tổng số dãy phủ “nhỏ nhất bên phải” lại không vượt quá 10.000; điều này làm quy mô thực tế của bài toán giảm mạnh. Vì vậy, đọc kỹ đề là bước đầu tiên để giải bài toán mở rộng quy mô.

Những bài toán ta giải trong thi tin học đều là sản phẩm của việc lý tưởng hóa bài toán thực tế, còn trong đời sống hiện thực, thứ thật sự cần xử lý và giải quyết là lượng dữ liệu rất lớn. Đây chính là một ý nghĩa thực tế quan trọng của việc hiện nay ta tiếp xúc với bài toán mở rộng quy mô; đồng thời loại bài toán ấy cũng đặt ra yêu cầu cao hơn cho chúng ta. Cần chú ý rằng bài toán mở rộng quy mô tuy quan trọng, nhưng không phải là lâu dài trên không: nó được xây dựng trên cơ sở các bài toán quy mô nhỏ. Vì vậy, muốn giải tốt bài toán mở rộng quy mô thì phải bắt đầu từ nền tảng; đạo lý ấy không chỉ thích hợp với thi tin học. Vừa có nền tảng lý thuyết vững chắc, vừa có năng lực thực hành và tinh thần sáng tạo, là yêu cầu cơ bản của thời đại đối với thanh niên bước sang thế kỷ mới, và cũng là mục tiêu phấn đấu không ngừng của chúng ta.

A Phụ lục

1. “Hạ số chiều” được nhắc trong bài này là một tư tưởng hoạch định chiến lược, có khác biệt nhất định với khái niệm hạ số chiều theo nghĩa thông thường.

B Chương trình

B.1 Chương trình cho bài toán một

Vì phương pháp liệt kê có hiệu suất quá thấp, ở đây chỉ đưa ra chương trình của thuật toán hiệu quả.

```
program object;
type
  node=record{luu so nguyen do chinh xac cao}
    v:array[1..100] of longint;
    last:integer;
  end;
var
  n,i:integer;
  a:array[1..10] of integer;{luu thong tin cua vat the n chieu}
  tot:node;{tong tich bac}

procedure readn;{doc thong tin cua vat the n chieu}
var i:integer;
    f:text;
begin
  assign(F,'input.txt');reset(F);
  readln(F,n);
  for i:=1 to n do read(f,a[i]);
  close(f);
end;

function num(s:integer):longint;
begin
  num:=(s+2)*(s+1)*s div 6;
end;

procedure multiply(var nd:node;s:longint);{nhân số do chính xác cao}
var c:longint;
    i:integer;
begin
  c:=0;
  with nd do begin
    for i:=1 to last do
      begin
        v[i]:=v[i]*s+c;
        c:=v[i] div 10;
        v[i]:=v[i] mod 10;
      end;
    while c<>0 do begin
      inc(last);
      v[last]:=c mod 10;
      c:=c div 10;
    end;
  end;
```

```

    end;
end;

begin
    readn;
    with tot do
        begin
            fillchar(v,sizeof(v),0);
            v[1]:=1;last:=1;
            end;{khởi tạo}
        for i:=1 to n do multiply(tot,num(a[i]));{tính tổng tích bậc}
        with tot do{in kết quả}
            for i:=last downto 1 do write(v[i]);
        writeln;
    end.

```

B.2 Chương trình cho bài toán hai

Vì phương pháp 1 có hiệu suất quá thấp và phương pháp 2 không hiện thực được, ở đây chỉ đưa ra chương trình tương ứng với phương pháp 3.

```

program number;
const inputfile='number.in';
      outputfile='number.out';
var
    f:text;
    n,m:longint;
    mb:longint;{lưu số hiện có nhiều ước thực sự nhất}
    fn:integer;{lưu số ước thực sự của số hiện có tốt nhất}
    s,t,i,j:longint;{s+1,t là điểm đầu và cuối khi thống kê theo đoạn}
    a:array[1..30000] of integer;
    {mỗi lần thống kê 30000 số liên tiếp, gồm cả chính mỗi số}

procedure readn;
begin
    assign(f,inputfile);
    reset(F);
    readln(f,n,m);
    close(F);
end;

function max(x,y:longint):longint;{trả về số lớn hơn trong x,y}
begin
    if x>y then max:=x else max:=y;
end;

procedure output;
begin

```

```

assign(F,outputfile);
rewrite(F);
writeln(f,mb);
close(F);
end;

begin
  readn;
  mb:=0; fn:=0;
  s:=n-1;
  while s<m do begin
    t:=s+30000;
    if t>m then t:=m;
    fillchar(a,sizeof(a),0);
    a[1]:=1;{de tien thong ke, dat 1 co hai uoc, deu la 1}
    for j:=1 to trunc(sqrt(t)) do {liet ke cac uoc khac nhau}
      for i:=max(j+1,(s+1) div j) to t div j do inc(a[i*j-s],2);
      {cac boi cua uoc can thong ke}
    for j:=trunc(sqrt(s+1)) to trunc(sqrt(t)) do inc(a[j*j-s]);
    {thong ke so chinh phuong}
    for i:=1 to t-s do{so sanh ket qua doan nay voi gia tri tot nhat}
      if a[i]>fn then begin
        fn:=a[i];
        mb:=i+s;
      end;
    s:=t;
  end;
  output;
end.

```

B.3 Chương trình cho bài toán ba

Phương pháp quy hoạch động chỉ dùng để thảo luận lý thuyết và không có tính khả thi.

```

{$A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R+,S+,T-,V+,X+}
{$M 16384,0,655360}
program chain;
const maxn=75;
      inputfile='input.txt';
      outputfile='output.txt';
type
  word=string[maxn];{định nghĩa kiểu tu}
var
  f:text;
  wd,prev,ans_wd:word;{wd: tu hiện tại; prev: tu trước; ans_wd: day dài nhất}
  now,max,i,l:integer;
  {now: day dài nhất gồm tu hiện tại; l: độ dài tu trước; max: độ dài tốt nhất}

```

```

procedure output(st:word);
{in xau st bang chu cai thuong}
var i:integer;
begin
  for i:=1 to length(st) do
    if st[i]=upcase(st[i]) then write(F,chr(ord(st[i])+32))
    else write(f,st[i]);
  writeln(F);
end;

begin
  assign(f,inputfile);reset(F);
  prev:='';{xoa tu truoc}
  l:=0;{dat do dai tu truoc bang 0}
  repeat
    readln(F,wd);{doc mot tu}
    if wd='.' then break;{dau hieu ket thuc tep}
    now:=0;{xoa so tu tien to trong day}
    for i:=1 to length(wd) do
      if i<=l then begin
        if prev[i]=upcase(wd[i]){co mot tu tien to}
        then begin inc(now);
          wd[i]:=upcase(wd[i]);{dung chu hoa danh dau vi tri tien to}
        end else if prev[i]<>wd[i] then break;
          {xuat hien khac biet voi tu truoc, thoat vong lap}
        end else break;{vuot do dai tu truoc, thoat vong lap}
    wd[length(wd)]:=upcase(wd[length(wd)]);{tu la tien to cua chinh no}
    l:=length(wd);{chuyen tiep cho tu sau}
    prev:=wd;
    if now>max then begin{giu lai day tu dai nhat}
      max:=now;
      ans_wd:=wd;
    end;
  until true=false;
  close(F);
  assign(F,outputfile);rewrite(f);
  {in ket qua}
  for i:=1 to length(ans_wd) do
    if ans_wd[i]=upcase(ans_wd[i]) then output(copy(ans_wd,1,i));
  close(F);
end.

```

B.4 Chương trình cho bài toán bốn

Với bài *Airport* của IOI 1999, trong chương trình `land_1.pas` thực ra cũng dùng điều kiện cắt tĩa. Vì hiệu quả không rõ nên ở phần chính không bàn kỹ. `land_3.pas` chỉ thêm hai điều kiện cắt tĩa trên cơ sở `land_2.pas`, các phần còn lại hoàn toàn giống nhau; vì vậy ở đây chỉ đưa ra `land_3.pas`. Văn bản trích xuất của PDF cũng chứa một bản `land_1.pas`; bản này được giữ lại trước `land_3.pas`.

```

{$A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R-,S+,T-,V+,X+}
{$M 2048,0,655360}
program land_1;
const maxn=700;
      last=100; {so cot doc mot lan, theo huong dong-tay}
      inputfile='land.inp';
      outputfile='land.out';
type
  arr=array[0..maxn] of integer;
  crr=array[0..10] of longint;
  maptype=array[1..maxn] of ^arr;
var
  f:text;
  u,v,c,i,j:integer;
  xmin,ymin,xmax,ymax:integer; {thong tin vung san bay}
  map:maptype;{ban do}
  area,maxarea:longint; {area: dien tich tot nhat hien tai; maxarea: can tren}
  Buf:array[1..8191] of Char; {bo dem 8K}
  len:array[1..maxn] of crr;
  wide:crr;

procedure get(left,right:integer); {doc cac cot tu left den right cua ban do}
var i,j,k,no:integer;
begin
  reset(F);
  for i:=v downto 1 do begin
    readln(F);
    for j:=1 to left-1 do read(f,k);
    for no:=left to right do
      if no<=u then read(f,map[no]^[i]) else break;
  end;
end;

procedure done(stl,edl:integer);
{tim vung lon nhat co bien tay la stl va bien dong khong qua edl}
var i,j,k,k0:integer;
    min,max:integer;
    maxw:longint;
begin
  if longint(edl-stl+1)*longint(v)<=area then exit;
  fillchar(len,sizeof(len),0);
  for i:=1 to v do
    for j:=0 to c do begin{vung tren hang co do cao trong [min,max]}
      k:=stl;
      min:=map[stl]^[i]-j;max:=min+c;
      repeat
        inc(k);
      until (k>edl) or (map[k]^[i]>max) or (map[k]^[i]<min);
    end;
  end;

```



```

        len[i,j]:=k-stl;
    end;
maxw:=v;
for i:=1 to v do begin
    dec(maxw); {chieu rong hien tai}
    for k:=0 to c do begin
        {tim dien tich lon nhat voi cao do thap nhat co dinh va chua dau hang i}
        wide[k]:=len[i,k]; j:=i;{bien bac}
        repeat
            if wide[k]*maxw<=area then break;
            if wide[k]*(longint(j)-longint(i)+1)>area then begin
                area:=wide[k]*(longint(j)-longint(i)+1);
                xmin:=stl;xmax:=xmin+wide[k]-1;
                ymin:=i;ymax:=j;
            end;
            inc(j); {cong don bien bac}
            if j>v then break;
            k0:=map[stl]^[j]-map[stl]^[i]+k;
            if (k0<0) or (k0>c) then break;
            if len[j,k0]<wide[k] then wide[k]:=len[j,k0];
        until true=false;
    end;
end;
end;

begin
    area:=1;
    xmin:=1;xmax:=1; ymin:=1;ymax:=1;
    {khởi tạo thông tin vùng}
    assign(f,inputfile);
    SetTextBuf(F, Buf);
    reset(F);
    readln(F,u,v,c);
    if 100>u then maxarea:=longint(v)*longint(u) else maxarea:=longint(v)*100;
    for i:=1 to 100 do new(map[i]);
    get(1,100);
    for i:=1 to u-99 do begin
        done(i,i+99);
        dispose(map[i]);{giải phóng bộ nhớ}
        if area=maxarea then break;{đã đạt đến diện tích lớn nhất, thoát vòng lặp}
        if (i mod last=1) and (i+last+99<=u) then begin{đọc dữ liệu vòng tiếp theo}
            for j:=i+100 to i+last+99 do new(map[j]);
            get(i+100,i+last+99);
        end;
    end;
    for i:=(u div last*last)+1 to u do new(map[i]);
    get((u div last*last)+1,u); {đọc phần dữ liệu còn lại}
    for i:=u-98 to u do if i>0 then done(i,u);

```

```

close(F);
assign(f,outputfile);rewrite(F);
writeln(f,area);
writeln(f,xmin,' ',ymin,' ',xmax,' ',ymax);
close(F);
end.

{$A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R-,S+,T-,V+,X+}
{$M 2048,0,655360}
program land_3;
const maxn=700;
    last=100;{so cot doc mot lan, theo huong dong-tay}
    inputfile='land.inp';
    outputfile='land.out';
type
    arr=array[0..maxn] of integer;
    maptype=array[1..maxn] of ^arr;
var
    f:text;
    u,v,c,i,j:integer;
    xmin,ymin,xmax,ymax:integer;{thong tin vung san bay}
    map:maptype;{ban do}
    min,max:array[1..maxn] of integer;{luu do cao nho nhat, lon nhat trong vung}
    area,maxarea:longint;{area: dien tich tot nhat hien tai; maxarea: can tren}
    Buf:array[1..8191] of Char; {bo dem 8K}

procedure get(left,right:integer);{doc cac cot tu left den right cua ban do}
var i,j,k,no:integer;
begin
    reset(F);
    for i:=v downto 1 do begin
        readln(F);
        for j:=1 to left-1 do read(f,k);
        for no:=left to right do
            if no<=u then read(f,map[no]^[i]) else break;
        end;
    end;
end;

procedure done(stl,edl:integer);
{tim vung lon nhat co bien tay la stl va bien dong khong qua edl}
var i,j,k,w,l:integer; {w: so o vuong cua vung hien tai theo huong dong-tay}
    minall,maxall:integer;
begin
    if longint(edl-stl+1)*longint(v)<=area then exit;
    for i:=1 to v do min[i]:=maxint;
    for i:=1 to v do max[i]:=-maxint;
    w:=0;{do dai theo huong dong-tay}
    l:=v;{chieu rong lon nhat theo huong nam-bac}

```

```

for i:=stl to edl do begin{bien dong cua vung}
  for j:=1 to v do begin
    if map[i]^j<min[j] then min[j]:=map[i]^j;
    {do cao nho nhat tren hang j tu cot stl den i}
    if map[i]^j>max[j] then max[j]:=map[i]^j;
    {do cao lon nhat tren hang j tu cot stl den i}
  end;
  inc(w);
  if longint(edl-stl+1)*longint(l)<=area then break;{dieu kien cat tia 1}
  if longint(w)*longint(l)<=area then continue; {dieu kien cat tia 2}
  {neu dien tich lon nhat co the dat van khong vuot qua dien tich hien tai}
  l:=0;{tinh lai gia tri L}
  for j:=1 to v do begin{bien bac cua vung}
    k:=j+1;{bien nam cua vung}
    minall:=maxint;maxall:=-maxint;
    repeat
      dec(k);
      if k=0 then break;
      if min[k]<minall then minall:=min[k];
      if max[k]>maxall then maxall:=max[k];
      {trong vung tu k den j, do cao nho nhat va lon nhat la minall,maxall}
    until maxall-minall>c;
    {vung thu duoc co khoang trai theo huong nam-bac la [k+1,j]}
    if j-k>l then l:=j-k;{cap nhat L}
    if longint(w)*longint(j-k)>area then begin
      area:=longint(w)*longint(j-k);
      xmin:=stl; xmax:=i;
      ymin:=k+1; ymax:=j;
    end;
  end;
end;
end;
end;

begin
  area:=1;
  xmin:=1;xmax:=1; ymin:=1;ymax:=1;
  {khởi tạo thông tin vùng}
  assign(f,inputfile);
  SetTextBuf(F, Buf);
  reset(F);
  readln(F,u,v,c);
  if 100>u then maxarea:=longint(v)*longint(u) else maxarea:=longint(v)*100;
  for i:=1 to 100 do new(map[i]);
  get(1,100);
  for i:=1 to u-99 do begin
    done(i,i+99);
    dispose(map[i]);{giai phóng bộ nhớ}
    if area=maxarea then break;{đã đạt đến diện tích lớn nhất, thoát vòng lặp}
  end;
end;

```

```

    if (i mod last=1) and (i+last+99<=u) then begin{doc du lieu vong tiep theo}
        for j:=i+100 to i+last+99 do new(map[j]);
        get(i+100,i+last+99);
    end;
end;
for i:=(u div last*last)+1 to u do new(map[i]);
get((u div last*last)+1,u); {docphan du lieu con lai}
for i:=u-98 to u do if i>0 then done(i,u);
close(F);
assign(f,outputfile);rewrite(F);
writeln(f,area);
writeln(f,xmin,' ',ymin,' ',xmax,' ',ymax);
close(F);
end.

```

C Tài liệu tham khảo

1. Wu Wenhui, Wang Jiande. *Phân tích thuật toán thực dụng và lập trình*. Nhà xuất bản Công nghiệp Điện tử.
2. Wu Wenhui, Wang Jiande. *Hướng dẫn thi Olympic Tin học quốc tế và quốc gia cho thanh thiếu niên: thuật toán và lập trình tổ hợp*. Nhà xuất bản Đại học Thanh Hoa.
3. Yan Weimin, Wu Weimin. *Cấu trúc dữ liệu*, tái bản lần thứ hai. Nhà xuất bản Đại học Thanh Hoa.